```
import tensorflow as tf
import cv2
import imghdr
import os
import tensorflow as tf
import os
import matplotlib.pyplot as plt
from matplotlib.image import imread
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten, Dropout
from tensorflow.keras.metrics import Precision, Recall, BinaryAccuracy
```

```
gpus = tf.config.experimental.list_physical_devices("GPU")
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)
```

```
data_dir = 'data'
image_exts = ['jpeg', 'jpg', 'bmp', 'png']
```

```
os.listdir(os.path.join(data_dir, 'sad'))
```

```
for image_class in os.listdir(data_dir):
    for image in os.listdir(os.path.join(data_dir, image_class)):
        print(image)
```

```
img = cv2.imread(os.path.join('data','happy','Screenshot 2024-06-30 at 10.59.57.png'))
```

```
img.shape
```

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.show()
```

```
for image_class in os.listdir(data_dir):
    for image in os.listdir(os.path.join(data_dir, image_class)):
        image_path = os.path.join(data_dir, image_class)
        try:
            img = cv2.imread(image_path)
            tip = imghdr.what(image_path)
            if tip not in image_exts:
                print('ímage not in ext list {}'.format(image_path))
                os.remove(image_path)

        except Exception as e:
            print('Issue with image {}'.format(image_path))
```

```
for image_class in os.listdir(data_dir):
    class_path = os.path.join(data_dir, image_class)
    if not os.path.isdir(class_path):
        continue  # Skip non-directory items

    for image in os.listdir(class_path):
        image_path = os.path.join(class_path, image)
        try:
            # Read the image using cv2
            img = cv2.imread(image_path)
            if img is None:
                print('Unable to read image: {}'.format(image_path))
                continue

            # Check the image type using imghdr
            tip = imghdr.what(image_path)
            if tip not in image_exts:
                print('Image not in expected format: {}'.format(image_path))
                os.remove(image_path)
                print('Removed:', image_path)

        except Exception as e:
            print('Error processing image {}: {}'.format(image_path, str(e)))
```

```
data = tf.keras.utils.image_dataset_from_directory('data')
```

```
data_iterator = data.as_numpy_iterator()
```

```
data_iterator
```

```
batch = data_iterator.next()
```

```
batch[0].shape
```

```
batch[1]
```

```
fig, ax = plt.subplots(ncols=4, figsize=(20,20))
for idx, img in enumerate(batch[0][:4]):
    ax[idx].imshow(img.astype(int))
    ax[idx].title.set_text(batch[1][idx])

#0 is happy
#1 is sad
```

```
scaled = batch[0] / 255
```

```
scaled.max()
```

```
data = data.map(lambda x,y: (x/255, y))
```

```
scaled_iterator = data.as_numpy_iterator()
```

```
scaled_iterator.next()[0].max()
```

```
len(data)
```

```
train_size = int(len(data)*.7)
val_size = int(len(data)*.2)+1
test_size = int(len(data)*.1)+1
```

```
train_size
```

```
val_size
```

```
test_size
```

```
train = data.take(train_size)
val = data.skip(train_size).take(val_size)
test = data.skip(train_size+val_size).take(test_size)

#shuffled data
```

```
model = Sequential()
```

```
model.add(Conv2D(16, (3,3), 1, activation='relu',input_shape=(256,256,3)))
model.add(MaxPooling2D())

model.add(Conv2D(32, (3,3), 1, activation='relu'))
model.add(MaxPooling2D())

model.add(Conv2D(16, (3,3), 1, activation='relu'))
model.add(MaxPooling2D())

model.add(Flatten())

model.add(Dense(256, activation='relu'))
model.add(Dense(1,activation='sigmoid'))
```

```
model.compile('adam', loss=tf.losses.BinaryCrossentropy(), metrics=['accuracy'])
```

```
model.summary()
```

```
#train
```

```
logdir = 'logs'
```

```
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=logdir)
```

```
hist = model.fit(train, epochs=20, validation_data=val, callbacks=[tensorboard_callback])
```

```
hist.history
```

```
fig = plt.figure()
plt.plot(hist.history['loss'], color='blue', label='loss')
plt.plot(hist.history['val_loss'], color='red', label='val_loss')
fig.suptitle('Loss', fontsize=20)
plt.legend(loc='upper left')
plt.show()
```

```
fig = plt.figure()
plt.plot(hist.history['accuracy'], color='blue', label='accuracy')
plt.plot(hist.history['val_accuracy'], color='red', label='val_accuracy')
fig.suptitle('Accuracy', fontsize=20)
plt.legend(loc='upper left')
plt.show()
```

```
#evaluate
```

```
pre = Precision()
re = Recall()
acc = BinaryAccuracy()
```

```
len(test)
```

```
for batch in test.as_numpy_iterator():
    x, y = batch
    yhat = model.predict(x)
    pre.update_state(y, yhat)
    re.update_state(y, yhat)
    acc.update_state(y, yhat)
```

```
print(f'Precision:{pre.result().numpy()}, Recall:{re.result().numpy()}, Accuracy:{acc.result().numpy()}')
```

```
#test
```

```
img = cv2.imread('happytest.png')
plt.imshow(cv2.cvtColor(img, cv2. COLOR_BGR2RGB))
plt.show()
```

```
resize = tf.image.resize(img, (256,256))
plt.imshow(resize.numpy().astype(int))
plt.show()
```

```
yhat = model.predict(np.expand_dims(resize/255, 0))
```

```
yhat
```

```
if yhat > 0.5:
    print(f'Predicted class is Happy')
else:
    print(f'Predicted class is Sad')
```

```
from tensorflow.keras.models import load_model
```

```
model.save(os.path.join('models', 'happysadmodel.h5'))
```

```
os.path.join('models', 'happysadmodel.h5')
```

```
new_model = load_model(os.path.join('models', 'happysadmodel.h5'))
```

```
new_model.predict(np.expand_dims(resize/255, 0))
```